



Graph based Clustering for Image Segmentation

COMP7404

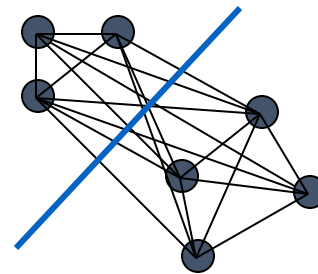
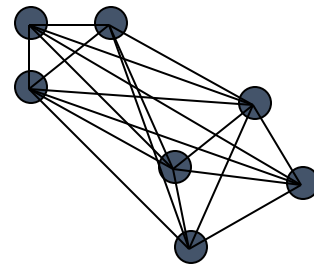
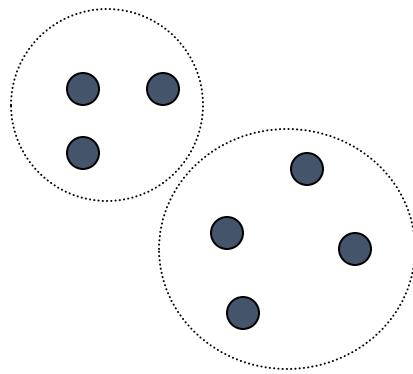
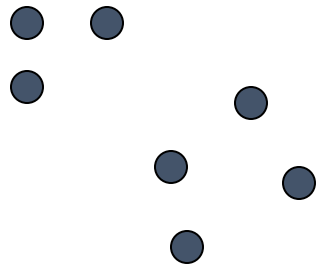
Computational Intelligence and Machine Learning

Dong Xu

Graph based clustering

- Associate each data item with a vertex in a weighted graph, where the weights on the edges between elements are large if the elements are similar and small if they are not.
- Cut the graph into connected components with relatively large interior weights by cutting edges with relatively low weights.
- Clustering becomes a graph cut problem.

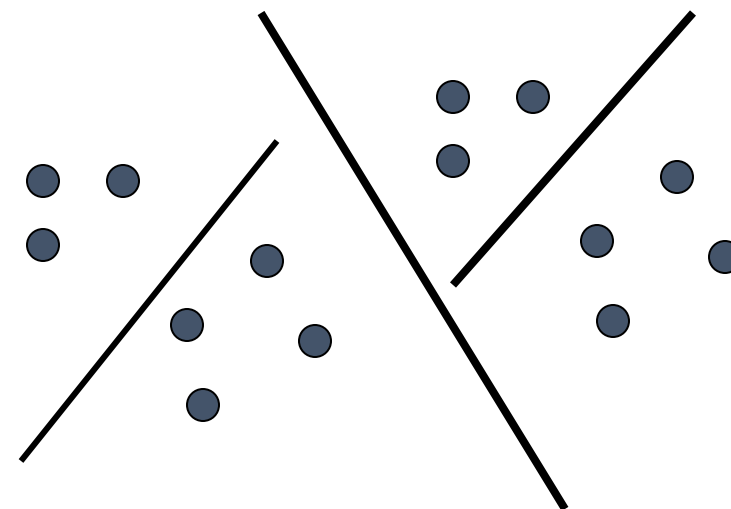
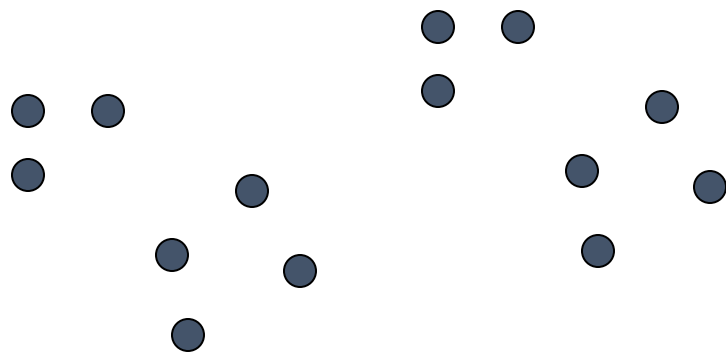
Example



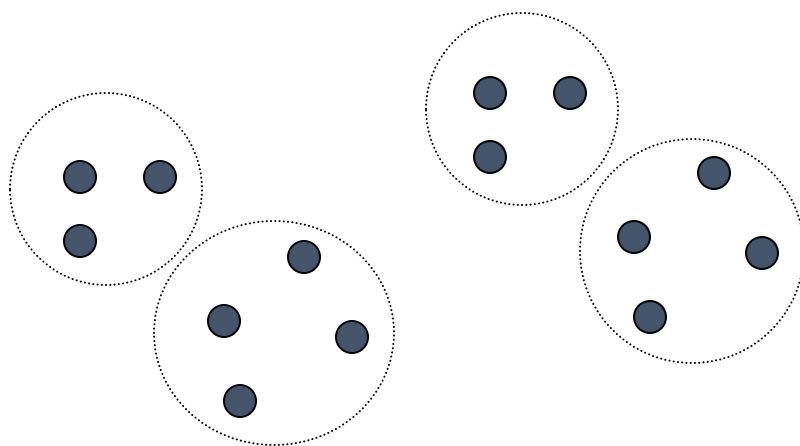
Two different clustering methods

- Method #1
 - Partition into two clusters
 - Use procedure recursively
- Method #2
 - Directly compute k-way partitioning

Illustration



Method #1



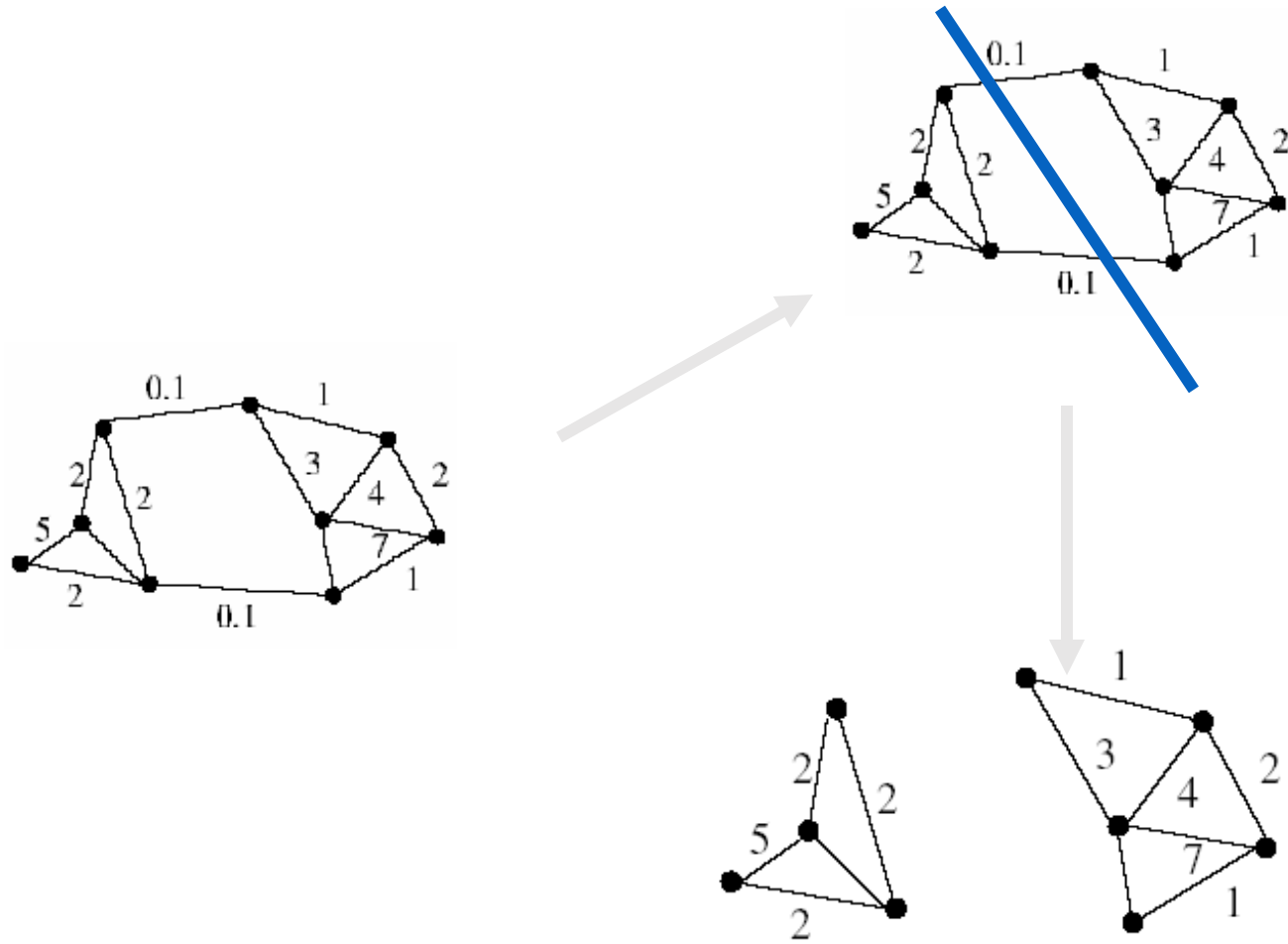
Method #2

Method #1: Graph cut

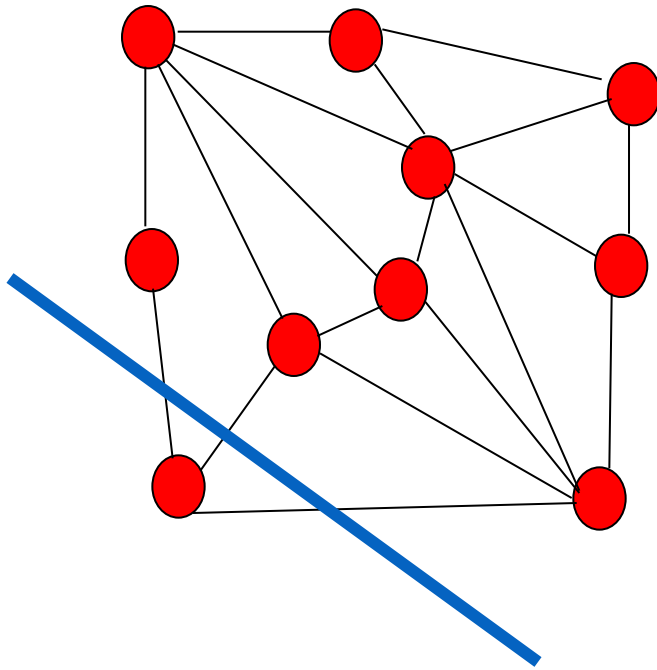
- A graph $G=(V,W,E)$ can be partitioned into two disjoint sets A, B , with $A \cap B = \emptyset$.
- The degree of dissimilarity between these two sets can be computed as total weight of the edges that connect these two sets. In graph theoretic language, it is called the cut:

$$Cut(A, B) = \sum_{u \in A, v \in B} w(u, v)$$

Illustration



Minimum Cut



A cut $\langle A, B \rangle$ of a graph G is the set of edges S that connects vertices of A to vertices of B .

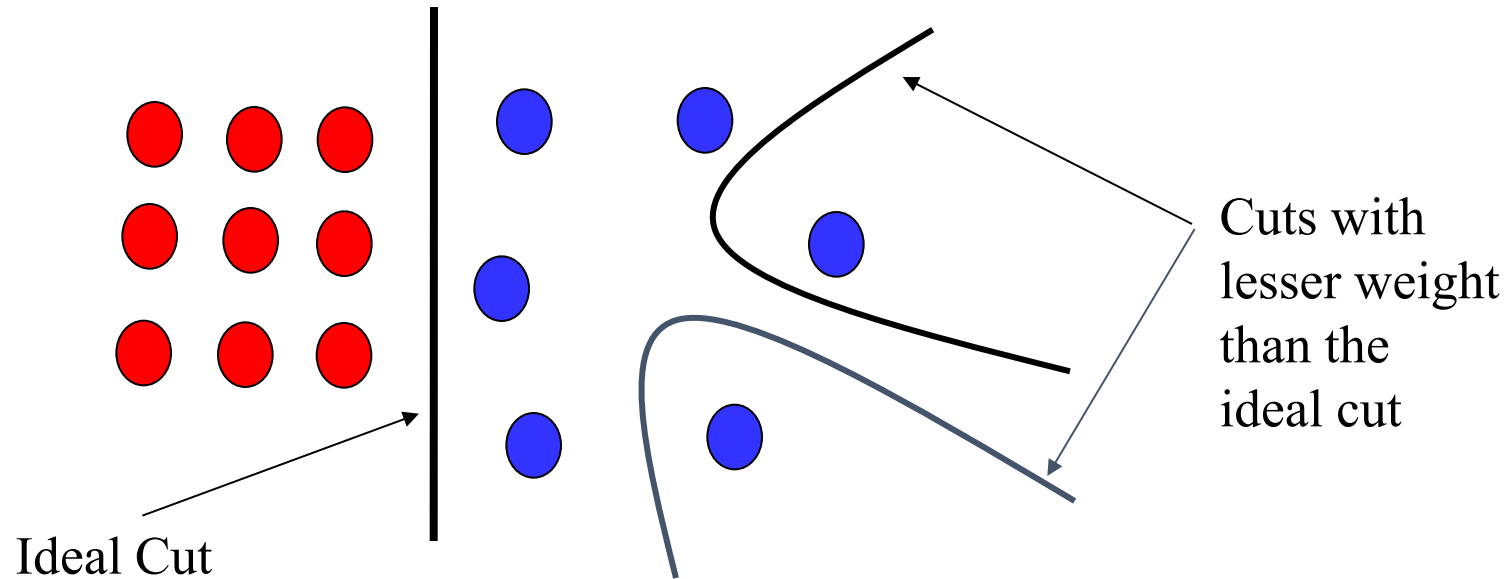
Minimum cut is the cut of minimum weight, where weight of cut $\langle A, B \rangle$ is given as

$$w(\langle A, B \rangle) = \sum_{x \in A, y \in B} w(x, y)$$

Wu and Leahy, PAMI'93

Drawbacks of Minimum Cut

- Weight of cut is directly proportional to the number of edges in the cut.



Normalized Cuts

- The normalized graph cuts algorithm (Shi & Malik, PAMI' 00) balances the sizes of the two clusters.
- Computes the cut cost as a fraction of the total edge connections to all the nodes in the graph. We call this the normalized cuts (Ncuts):

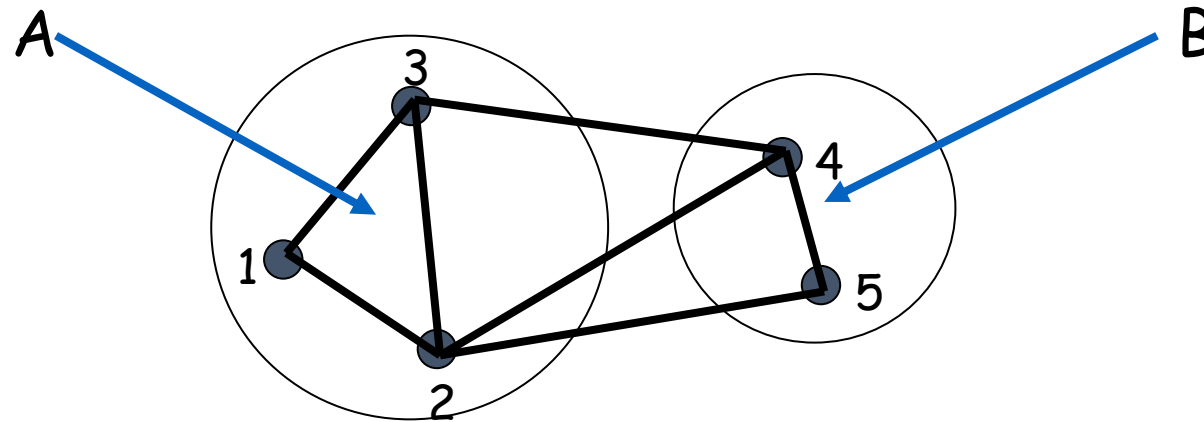
$$Ncuts(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}$$

$$assoc(A, V) = \sum_{u \in A, t \in V} w(u, t)$$

Normalized Cuts (Cont'd)

$$Ncuts(A, B) = \frac{cut(A, B)}{assoc(A, V)} + \frac{cut(A, B)}{assoc(B, V)}$$

$$assoc(A, V) = \sum_{u \in A, t \in V} w(u, t)$$



$$Cut(A, B) = w(3, 4) + w(2, 4) + w(2, 5)$$

$$Assoc(A, V) = w(1, 3) + w(1, 2) + w(2, 3) + w(3, 4) + w(2, 4) + w(2, 5)$$

$$Assoc(B, V) = w(4, 5) + w(3, 4) + w(2, 4) + w(2, 5)$$

W,D, and y

0	0.3	0.5	0.4	0.7
0.3	0	0.4	0.6	0.5
0.5	0.4	0	0.8	0.6
0.4	0.6	0.8	0	0.7
0.7	0.5	0.6	0.7	0

1.9
1.8
2.3
2.5
2.5

1.9	0	0	0	0
0	1.8	0	0	0
0	0	2.3	0	0
0	0	0	2.5	0
0	0	0	0	2.5

W: similarity (affinity) matrix

D

V1: A
V2: A
V3: A
V4: B
V5: B



$$y = \begin{pmatrix} 1 \\ 1 \\ 1 \\ -b \\ -b \end{pmatrix}$$

Simplification of the criterion

- Write a vector y whose elements are 1 if the item is in A, -b if it's in B
- Write the matrix of the graph as W , and the matrix which has the row sums of W on its diagonal as D , $\mathbf{1}$ is the vector with all ones.

- Criterion becomes $\min_y \left(\frac{y^T (D - W) y}{y^T D y} \right)$ *Laplacian Matrix:*
 $L = D - W$

- Constraint $y^T D \mathbf{1} = 0$

- NP-complete

Simplification of the criterion

- Instead, relax into the continuous domain by solving generalized eigenvalue system:

$$\min_y (y^T (D - W)y) \text{ subject to } (y^T D y = 1)$$

- Which gives: $(D - W)y = \lambda Dy$
- Note that $(D - W)1 = 0$, so the first eigenvector is $y_0=1$ with eigenvalue 0.
- The second smallest eigenvector is the real valued solution to this problem!!

Algorithm

1. Define a similarity function between 2 nodes. i.e.:

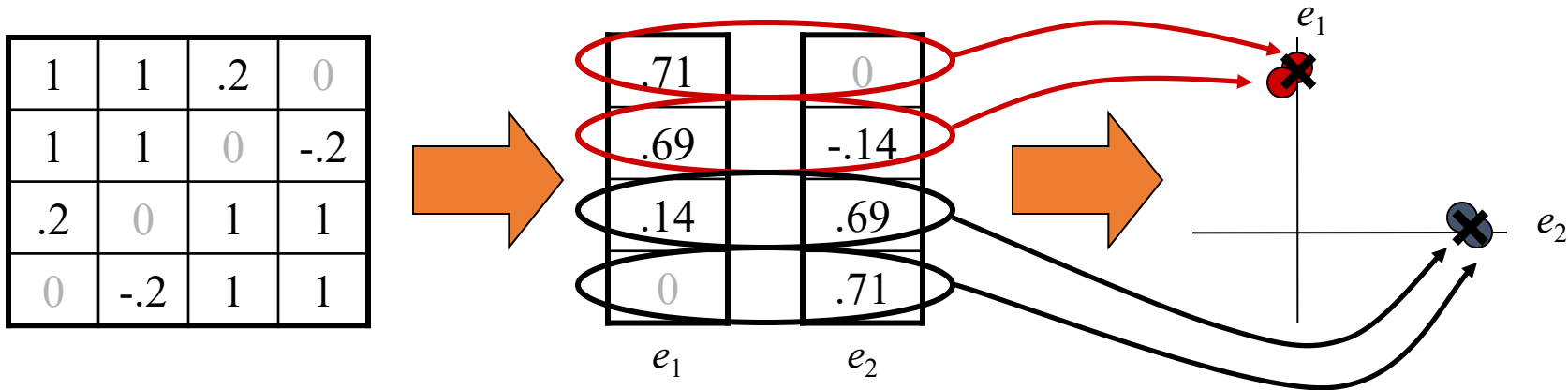
$$w_{i,j} = e^{\frac{-\|F_{(i)} - F_{(j)}\|_2^2}{\sigma_I^2} + \frac{-\|X_{(i)} - X_{(j)}\|_2^2}{\sigma_X^2}}$$

2. Compute affinity matrix (W) and degree matrix (D).
3. Solve $(D - W)y = \lambda Dy$
4. Use the eigenvector with the second smallest eigenvalue to bipartition the graph.
5. Decide if re-partition current partitions.

Note: since precision requirements are low, W is very sparse and only few eigenvectors are required, the eigenvectors can be extracted very fast using Lanczos algorithm.

Use k -eigenvectors

- Recursive 2-way *Ncut* is slow.
- We can use more eigenvectors to re-partition the graph, however:
 - Not all eigenvectors are useful for partition.
- Procedure: compute k -means with a high k . Then follow one of these procedures:
 - a) Merge segments that minimize k -way *Ncut* criterion.
 - b) Use the k segments and find the partitions there using exhaustive search.



Experiments

$$w_{i,j} = e^{\frac{-\|F_{(i)} - F_{(j)}\|_2^2}{\sigma_I^2} + \frac{-\|X_{(i)} - X_{(j)}\|_2^2}{\sigma_X^2}}$$

- Define similarity:

- for point sets. $F(i) = 1$
- for brightness images. $F(i) = I(i)$
- for HSV images. $F(i) = [v, v.s.\sin(h), v.s.\cos(h)]$
- in case of texture. $F(i) = [|I * f_1|, \dots, |I * f_n|]$

Results on image segmentation



(a)



(b)



(c)



(d)



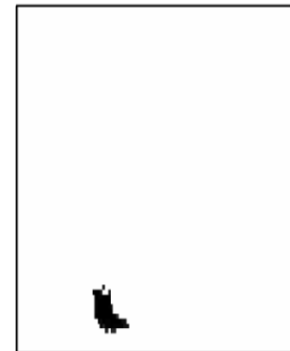
(e)



(f)



(g)



(h)

Thank you!